

A PROJECT REPORT ON

AI-BASED SMART ATTENDANCE

MANAGEMENT SYSTEM

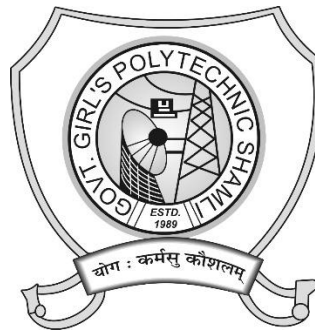
Submitted in partial fulfilment of the requirements for the award of the diploma of

DIPLOMA IN COMPUTER SCIENCE ENGINEERING

Submitted by

ANKITA PANDEY

4th Semester, Computer Science Engineering



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

GOVERNMENT GIRLS POLYTECHNIC, SHAMLI

2026

CERTIFICATE

This is to certify that the project report entitled “AI-Based Smart Attendance Management System” is a bonafide record of the project work carried out by Ankita Pandey, a student of the Diploma in Computer Science Engineering programme at Government Girls Polytechnic, Shamli, during the academic session 2026-27, in partial fulfilment of the requirements for the award of the Diploma in Computer Science Engineering.

The project was carried out under my guidance and supervision. To the best of my knowledge, the work reported here does not form a part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion, and the matter embodied represents the student's own independent work.

Date:

Place: Shamli, Uttar Pradesh

Head of Department

Department of Computer Science Engineering

DECLARATION

I, Ankita Pandey, a student of the Diploma in Computer Science Engineering programme at Government Girls Polytechnic, Shamli, hereby declare that the project report entitled “AI-Based Smart Attendance Management System”, submitted in partial fulfilment of the requirements for the award of the Diploma in Computer Science Engineering, is an authentic record of my own work carried out under the guidance of Mr. Vivek Kumar.

The matter presented in this report has not been submitted by me, in part or full, for the award of any other degree or diploma of this or any other institute.

Ankita Pandey

4th Semester, Computer Science Engineering

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Mr. Vivek Kumar, for his constant guidance, patience and encouragement throughout the design, development and documentation of this project. His insistence on sound architectural practice — layered separation of concerns, row-level data security, and an auditable trail for every sensitive action — shaped this project far beyond what a college assignment would ordinarily demand.

I am grateful to the Head of the Department of Computer Science Engineering and to the Principal of Government Girls Polytechnic, Shamli, for providing the infrastructure, laboratory access and encouragement necessary to build and test a system of this scale.

I also thank the faculty members of the department who patiently answered questions about the college's real academic structure, timetable conventions and attendance policy, which allowed the system to be seeded with genuine institutional data rather than placeholder examples.

Finally, I thank my family and friends for their continuous support and encouragement during the course of this project.

Ankita Pandey

ABSTRACT

Attendance in most classrooms in India is still recorded on paper: a register is passed around, a teacher calls out names, and the results are transcribed by hand into a semester-end summary. The process is slow, difficult to audit, trivially exploited through proxy attendance, and gives neither the teacher nor the student any real-time picture of where they stand against the mandatory attendance threshold. This project, the AI-Based Smart Attendance Management System, was built to replace that process for Government Girls Polytechnic, Shamli, with a offline, desktop application that automates attendance capture using face recognition while retaining every manual control a real college actually needs.

The system offers three complementary ways of marking attendance that all converge on a single database record: automatic marking through live face recognition (the primary route), one-click manual marking and correction by the subject teacher, and a supervised self-marking route in which a student's own tap only raises a pending request that must be approved by the teacher before it counts. Built around three roles — Administrator, Faculty and Student — the application additionally manages the full academic structure (courses, branches, semesters, sections, subjects and timetable).

Face recognition is implemented with a dual-backend strategy: the high-accuracy dlib/face_recognition embedding model is used automatically when it is installed, and the system falls back transparently to OpenCV's LBPH recognizer — which needs no C++ compiler and runs on any ordinary college computer — when it is not. Both backends expose an identical, normalised 0–100% confidence interface to the rest of the application. The system is implemented in Python using CustomTkinter for the interface and SQLite (in Write-Ahead Logging mode) for storage, spans 56 modules and roughly 23,000 lines of code across a clearly layered architecture — presentation, service, business-logic and core/infrastructure — backed by a 24-table relational schema.

The result is a system that a real institution can install and use offline on day one — it seeds itself with the college's own published academic structure and staff directory on first run — while remaining auditable, secure and extensible enough to accommodate more branches, more sections and, eventually, a cloud-synchronised or mobile companion in future work.

TABLE OF CONTENTS

CERTIFICATE	2
DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES	9
LIST OF TABLES	10
CHAPTER 1: INTRODUCTION	11
1.1 Background	11
1.2 Problem Statement	11
1.3 Objectives	12
1.4 Scope of the Project	13
1.5 Organisation of the Report.....	13
CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM STUDY	14
2.1 Manual Attendance Systems	14
2.2 Existing Automated Alternatives and Their Limitations	14
2.3 Why Face Recognition.....	14
2.4 Comparative Analysis	15
2.5 Proposed System Overview	15
CHAPTER 3: SYSTEM ANALYSIS.....	16
3.1 Functional Requirements	16
3.2 Non-Functional Requirements	17
3.3 Feasibility Study	18
3.4 Hardware and Software Requirements	19
CHAPTER 4: SYSTEM DESIGN.....	20

4.1 System Architecture.....	20
4.2 Data Flow Diagrams	21
4.3 Use Case Diagram.....	22
4.4 Database Design.....	23
4.5 Key Architectural Decisions	24
CHAPTER 5: TECHNOLOGY STACK.....	25
5.1 Programming Language and GUI Framework	25
5.2 Face Recognition Backends.....	25
5.3 Supporting Libraries	26
5.4 Database	26
5.5 Development and Version Control	26
CHAPTER 6: IMPLEMENTATION	27
6.1 Module-wise Implementation Summary.....	27
6.2 The Face Recognition Pipeline	27
6.3 The Three Attendance Routes.....	29
6.4 Leave Management Workflow	30
6.5 Security Implementation.....	31
6.6 Reports and Analytics	32
6.7 Representative User Interface	33
CHAPTER 7: TESTING.....	40
7.1 Testing Approach.....	40
7.2 Representative Test Cases.....	40
7.3 Automated Smoke-Test Suite	41
7.4 Results Summary	42
CHAPTER 8: RESULTS AND DISCUSSION.....	43
8.1 Screens Delivered	43
8.2 Achievements Against Objectives	43

8.3 Limitations	44
8.4 Discussion	44
CHAPTER 9: CONCLUSION AND FUTURE SCOPE.....	45
9.1 Conclusion	45
9.2 Future Scope	45
REFERENCES	47
APPENDIX A: COMPLETE DATABASE SCHEMA.....	48
APPENDIX B: REPRESENTATIVE CODE EXCERPTS.....	50
B.1 Normalised Confidence Scoring (services/face_service.py).....	50
B.2 Pre-Seeding an Attendance Session (models/attendance.py).....	50
B.3 Role-Based Permission Check (core/auth.py).....	50
B.4 Self-Marking Never Mutates Attendance Directly (models/self_attendance.py)	50
APPENDIX C: GLOSSARY OF TERMS	52

LIST OF FIGURES

Fig. 4.1	Layered System Architecture
Fig. 4.2	Data Flow Diagram — Level 0 (Context)
Fig. 4.3	Data Flow Diagram — Level 1
Fig. 4.4	Entity-Relationship Diagram (core entities).....
Fig. 4.5	Use Case Diagram
Fig. 6.1	Sequence Diagram — Face-Recognition Attendance
Fig. 6.2	Sequence Diagram — Student Self-Marking with Verification.....
Fig. 6.3	Three Complementary Attendance Routes.....
Fig. 6.4	Sign-In Screen
Fig. 6.5	Administrator Dashboard
Fig. 6.6	Faculty Dashboard.....
Fig. 6.7	Student Dashboard.....
Fig. 6.8	Face-Recognition Attendance Screen.....
Fig. 6.9	Manual Attendance and Correction.....
Fig. 6.10	Attendance Verification Queue
Fig. 6.11	Student Management Screen
Fig. 6.12	Weekly Timetable.....
Fig. 6.13	Reports Screen.....
Fig. 6.14	Analytics Screen
Fig. 6.15	Audit Trail

LIST OF TABLES

Table 2.1	Comparison of Attendance-Automation Technologies
Table 3.1	Functional Requirements
Table 3.2	Non-Functional Requirements
Table 3.3	Hardware and Software Requirements
Table 4.1	Database Table Groups
Table 5.1	Core Technology Stack.....
Table 5.2	Face Recognition Backend Comparison.....
Table 6.1	Module-wise Implementation Summary
Table 6.2	Role-wise Permission Matrix (representative)
Table 6.3	Report Catalogue
Table 7.1	Representative Test Cases
Table 9.1	Objectives vs. Achievements

CHAPTER 1: INTRODUCTION

1.1 Background

Attendance recording is one of the most repeated administrative tasks in any educational institution, and yet it remains, in most polytechnic colleges, entirely manual. At Government Girls Polytechnic, Shamli — the institution this project was built for — attendance across three diploma programmes (Computer Science & Engineering, Electronics Engineering, and Fashion Designing & Garment Technology) has traditionally been recorded in paper registers, one per subject per section, maintained by the subject teacher and consolidated by hand at the end of each month and semester.

This manual process has three well-known failure modes. First, it is slow: calling out fifty to sixty names by roll number consumes several minutes of every class period. Second, it is easy to defeat: a present student can mark an absent friend as present (proxy attendance), and a paper register offers no way to detect this after the fact. Third, it is opaque: neither the student nor the teacher has an easy, up-to-date view of cumulative attendance percentage, so a student who has quietly fallen below the mandatory 75% threshold typically finds out only close to the examination-eligibility deadline, when little can be done about it.

Automating this process is not simply a matter of digitising the register. A classroom automation system has to accommodate real institutional workflows that a purely automatic system would break: a teacher must still be able to correct a wrong mark, a student who forgets to be recognised by a camera must still have a legitimate way to be marked present, a student who is genuinely on medical leave must not be penalised, and every one of these exceptions must be traceable to who made it, when, and why. This is the specific problem this project addresses.

1.2 Problem Statement

To design and implement an offline, desktop attendance management system for a polytechnic college that: (a) automates day-to-day attendance capture using face recognition so that it cannot be defeated by proxy marking, (b) preserves a manual marking and correction path for the subject teacher for the cases automation cannot handle, (c) allows a student to raise their own attendance for teacher verification rather than trusting a self-report outright, (d) manages the full academic lifecycle around attendance — courses, branches, semesters, timetable, leave

and promotion — and (e) does all of this with role-based security, a complete audit trail, and reporting good enough for an accreditation or inspection visit, without depending on an internet connection or a College Management cloud subscription.

1.3 Objectives

The specific objectives set for the project were:

- To build a face-recognition attendance pipeline that runs on ordinary classroom hardware (a laptop webcam) without requiring a paid cloud API or specialised biometric hardware.
- To make proxy attendance structurally difficult rather than merely discouraged, by tying the primary marking route to a live camera feed and a database uniqueness constraint on (session, student).
- To provide a manual marking and correction workflow for teachers that never fights the automatic one, and that leaves a mandatory, audited reason whenever a face-recognised mark is overridden.
- To let students raise their own attendance without ever letting a student's own action, by itself, change an attendance record — every self-marked request must be approved by the class's own subject teacher.
- To model the complete academic structure — courses, branches, semesters, sections, subjects, timetable and holidays — as configurable data rather than hard-coded constants, so the system is not tied to one college's specific programme list.
- To provide role-based access control (Administrator, Faculty, Student) with row-level data scoping, so that a faculty member can only ever see and act on their own subjects and classes.
- To generate the reports a real institution needs — daily, weekly, monthly, semester, defaulter, leave and medical-leave reports in PDF, Excel and CSV — carrying the college's own letterhead, filters and generation timestamp.
- To maintain a tamper-evident audit trail of every sensitive action (attendance edits, leave overrides, lock/unlock, backup/restore, password resets) with before/after values and a mandatory reason where applicable.

1.4 Scope of the Project

The system is a single-institution, offline desktop application. It is explicitly scoped to:

- Run entirely on a local machine against a local SQLite database — no server, no internet dependency, no cloud account.
- Support the three roles actually present in a polytechnic college — Administrator, Faculty and Student — rather than a generic enterprise role hierarchy.
- Recognise faces captured by a standard USB or built-in laptop webcam under normal classroom lighting; it does not attempt liveness detection or anti-spoofing against a printed photograph, which is left as future work (Section 9.2).
- Seed itself, on first run, with Government Girls Polytechnic, Shamli's own published courses, branches, faculty directory and holiday calendar, so the system is demonstrably usable with real institutional data rather than placeholder examples.

Explicitly out of scope for the current version: a mobile companion application, multi-institution or multi-campus deployment, cloud synchronisation between multiple installations, SMS/email notifications, and biometric modalities other than face recognition (fingerprint, iris, etc.). These are discussed as future scope in Chapter 9.

1.5 Organisation of the Report

Chapter 2 reviews manual and existing automated attendance approaches and positions the proposed system against them. Chapter 3 presents the functional and non-functional requirements and a feasibility assessment. Chapter 4 covers system design — architecture, data flow, use cases and the database schema. Chapter 5 documents the technology stack. Chapter 6 is the implementation chapter: the module-wise breakdown, the face-recognition pipeline in detail, the three attendance routes, the leave workflow, security implementation, and representative screens. Chapter 7 covers the testing approach and representative test cases. Chapter 8 discusses results against the stated objectives and known limitations. Chapter 9 concludes the report and outlines future scope. References and two appendices — the complete database schema and representative code excerpts — follow.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM STUDY

2.1 Manual Attendance Systems

The paper register remains the dominant attendance method in Indian polytechnic colleges. Its advantages — zero cost, zero training, complete institutional familiarity — are real, but so are its costs. A teacher loses several minutes of every class period to roll call. Consolidating a semester's attendance percentage by hand across dozens of subjects is tedious and error-prone. Most importantly, a paper register has no way to prevent or even detect proxy marking: once a name is ticked present, there is no record of who actually ticked it or on what basis.

2.2 Existing Automated Alternatives and Their Limitations

A number of automated alternatives are already in institutional use elsewhere, each with trade-offs that make them a poor fit for a small polytechnic college running on a limited budget:

- RFID / smart-card attendance: requires issuing and maintaining a physical card per student, plus card readers at every classroom door. Cards can be lent to another student, which reintroduces proxy attendance through a different mechanism.
- Fingerprint biometric attendance: harder to share than a card, but scanners are a recurring hardware cost per classroom, degrade with dust and humidity, form a queue at the start of every period, and raise hygiene concerns in a shared academic setting.
- OTP / mobile-app self-check-in: depends on every student owning a smartphone and having reliable data connectivity, which cannot be assumed uniformly across a government polytechnic's student body, and is self-reported by construction unless paired with a second verification mechanism.

2.3 Why Face Recognition

Face recognition avoids the specific weaknesses above: it needs no card or token that can be lent to someone else, no per-classroom hardware beyond a webcam most laptops already have, no network connection, and no queue — a class of fifty students can, in principle, all be recognised within the time it takes the teacher to open the attendance screen. Its own weaknesses — sensitivity to lighting and camera angle, and the possibility of an unrecognised face — are the reason this project does not rely on face recognition alone; Sections 6.3 and 6.4

describe the manual and self-marking routes that exist specifically to absorb those failure cases without reintroducing proxy attendance.

2.4 Comparative Analysis

Attendance Method	Extra Hardware	Proxy-Resistant	Works Offline	Recurring Cost
Paper register	None	No	Yes	Stationery only
RFID card	Reader + cards	No (can be lent)	Yes	Cards readers +
Barcode / QR	Printer/scanner	No (can be shared)	Yes	Low
Fingerprint biometric	Fingerprint scanner	Partially	Yes	Scanner upkeep
Mobile OTP self-check-in	Student smartphones	No (self-reported)	No	SMS/data
Face recognition (proposed)	Webcam (usually built-in)	Yes	Yes	None

Table 2.1: Comparison of Attendance-Automation Technologies

2.5 Proposed System Overview

The proposed AI-Based Smart Attendance Management System combines face recognition as the primary attendance route with two deliberate fallbacks — one-click manual marking by the teacher, and a supervised self-marking route for students — so that the system degrades gracefully rather than failing outright when the camera route cannot be used for a particular student or session. It further extends beyond attendance capture into the surrounding academic workflow — academic structure management, leave, timetable, reporting and analytics — because attendance data is only useful when it can be read against a subject, a semester, an eligibility threshold and a leave record. The complete role model, architecture and data design are presented in the chapters that follow.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

ID	Requirement	Primary Actor(s)
FR-1	The system shall authenticate a user by username, password and a selected role, rejecting the attempt if the role does not match the account even when the password is correct.	All
FR-2	The system shall lock an account for a configurable period after five consecutive failed login attempts.	System
FR-3	The system shall let a user reset their own password by verifying identity against their registered mobile number or date of birth, without requiring an email server.	All
FR-4	The system shall let the Administrator define courses, branches, semesters, sections, academic sessions, batches and holidays.	Administrator
FR-5	The system shall let the Administrator and Faculty manage student and faculty records, including bulk import/export via Excel.	Administrator, Faculty
FR-6	The system shall capture a face dataset for a student, either from a live camera or from existing photographs, and train a recognition model from it.	Faculty, Administrator
FR-7	The system shall open a face-recognition attendance session for a selected subject and section, recognise enrolled students from a live camera feed, and mark them Present or Late automatically with a confidence score.	Faculty
FR-8	The system shall let a Faculty member mark or correct attendance manually, requiring a written reason whenever a face-recognised mark is changed.	Faculty
FR-9	The system shall let a student raise a request to be marked present for a scheduled class, which shall not itself alter the attendance record until a Faculty member approves it.	Student, Faculty
FR-10	The system shall let the Administrator lock or unlock an attendance session, requiring a reason for unlocking.	Administrator
FR-11	The system shall let a student apply for leave of a defined type, mandating a medical certificate beyond a configurable number of days for medical leave, and let the class's own faculty review it.	Student, Faculty

ID	Requirement	Primary Actor(s)
FR-12	The system shall let the Administrator override any leave decision with a mandatory reason.	Administrator
FR-13	The system shall detect timetable clashes across class, faculty and room before a slot is saved.	Administrator
FR-14	The system shall generate attendance, defaulter and leave reports in PDF, Excel and CSV formats, carrying the college's letterhead and the generating user's name.	All
FR-15	The system shall present attendance analytics — daily, weekly, monthly trends and branch/subject/faculty/semester comparisons — to each role appropriate to what they are permitted to see.	All
FR-16	The system shall record every sensitive action to an audit log with before/after values, and shall never allow a failure to write that log to roll back the underlying transaction.	System
FR-17	The system shall take automatic and on-demand backups of the database and allow a validated restore.	Administrator
FR-18	The system shall generate a printable student ID card carrying a Code 39 barcode of the enrolment number.	Administrator

Table 3.1: Functional Requirements

3.2 Non-Functional Requirements

Category	Requirement
Performance	Face recognition on a live frame should complete quickly enough to feel real-time on ordinary laptop hardware; the capture and recognition loop runs on a background worker thread so the interface never freezes.
Security	Passwords are hashed with bcrypt (cost factor 12); every screen and every underlying query is gated first by role-based permissions and then by row-level data scope; sensitive actions require a written reason and are audited.
Reliability	The database runs in Write-Ahead Logging mode so a background write (e.g. a face-recognition mark) does not block a foreground read; automatic backups run on a configurable schedule with pruning of old copies.

Category	Requirement
Usability	A single role-appropriate dashboard, a consistent light/dark theme, and one-click demo-credential fill on the login screen keep the day-to-day learning curve low for non-technical office and teaching staff.
Portability	The application is pure Python plus a small set of pip-installable dependencies; it runs unmodified on any Windows PC with Python 3.12–3.14 installed, with no server-side component.
Maintainability	The codebase is layered (presentation / service / business logic / core) with the academic structure modelled as data rather than constants, so adding a branch, a section or a leave type never requires a source change.
Auditability	Every login, attendance edit, leave decision, backup, restore and data import/export is logged with the acting user, timestamp and, where relevant, the reason given.

Table 3.2: Non-Functional Requirements

3.3 Feasibility Study

3.3.1 Technical Feasibility

All components required are free, open-source and well documented: Python, CustomTkinter for the interface, OpenCV for face detection and the LBPH recogniser, SQLite for storage, and ReportLab/OpenPyXL for reporting. No component requires a paid licence or a cloud subscription. The one technically demanding dependency — the higher-accuracy dlib/face_recognition backend — needs a C++ toolchain that is not always available on a college PC; the project addresses this directly by shipping OpenCV's LBPH recogniser as a fully-functional fallback that needs no compiler, auto-selected at start-up (Section 5.2).

3.3.2 Economic Feasibility

The only hardware requirement beyond an existing office/lab computer is a webcam, which is built into essentially every laptop sold in the last decade. This compares favourably with RFID (per-card and per-reader cost) or fingerprint biometrics (per-scanner cost and periodic replacement), both of which impose a recurring hardware budget that a government polytechnic would otherwise have to justify separately every academic year.

3.3.3 Operational Feasibility

The system is designed to fit around existing staff workflows rather than replace them outright: a teacher who prefers to mark attendance by hand can still do so from the Manual Attendance screen, and nothing in the system requires every student to be enrolled in the face-recognition dataset before the system can be used at all. First-run seeding of the college's own real academic structure and staff directory (Section 6.1) further reduces the operational burden of adopting the system, since the office does not need to re-enter the institution's basic data from scratch.

3.4 Hardware and Software Requirements

Component	Minimum Specification
Processor	Intel Core i3 (or equivalent), dual-core, 2.0 GHz or faster
RAM	4 GB (8 GB recommended for smoother face-training on large datasets)
Storage	2 GB free disk space for the application, database, photos and backups
Camera	Any built-in or USB webcam supported by DirectShow (used for face capture and recognition)
Operating System	Windows 10 / 11 (64-bit)
Runtime	Python 3.12 – 3.14
Key Python packages	customtkinter, opencv-contrib-python (<5), Pillow, numpy, pandas, matplotlib, openpyxl, reportlab, bcrypt, tkcalendar

Table 3.3: Hardware and Software Requirements

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The application is organised into four layers plus a data layer, each depending only on the layer below it. The presentation layer (ui/) contains the splash screen, the login screen, the main application shell with its permission-filtered sidebar, sixteen screen modules and a small set of reusable widgets (data tables, dialogs, cards). The service layer (services/) contains the pieces of business logic heavy enough to warrant their own module — face recognition, analytics chart generation, PDF/Excel/CSV report generation, ID-card generation and Excel import/export. The business logic layer (models/) is the repository layer: one module per domain area (academic structure, students, faculty, subjects, timetable, attendance, self-attendance, leave), each owning its own validation rules and SQL. The core/infrastructure layer (core/ and config/) provides the database connection singleton, authentication and RBAC, the audit and activity logs, backup/restore, input validators, row-level data scoping, and the persisted configuration and theme. Figure 4.1 shows this layering.

Fig. 4.1 Layered System Architecture

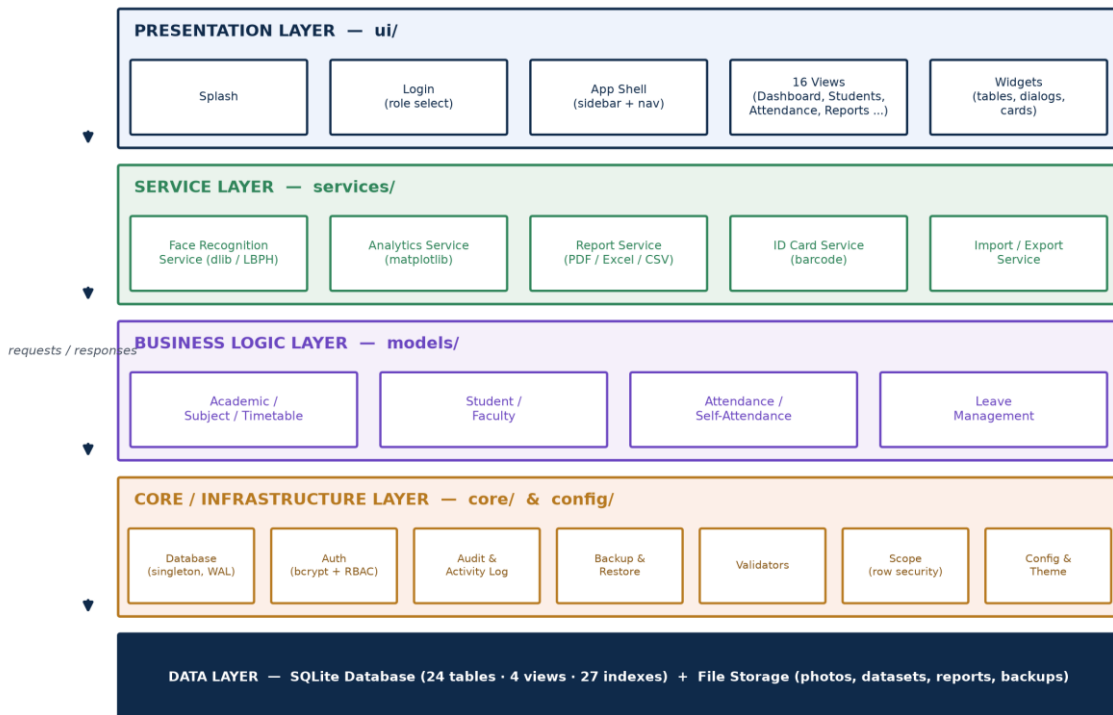


Fig. 4.1 Layered System Architecture

A view never talks to the database directly; it always goes through a model function, which in turn goes through the `core.database` singleton. This means a row-security rule added in `core/scope.py`, or a validation rule added in `core/validators.py`, is automatically enforced everywhere that model function is called, rather than needing to be repeated in every screen that happens to touch that data.

4.2 Data Flow Diagrams

Figure 4.2 shows the system in context: the four external entities that exchange data with it are the Administrator, Faculty and Student user roles, and the camera hardware that supplies live video frames during face-recognition attendance.

Fig. 4.2 Data Flow Diagram — Level 0 (Context)

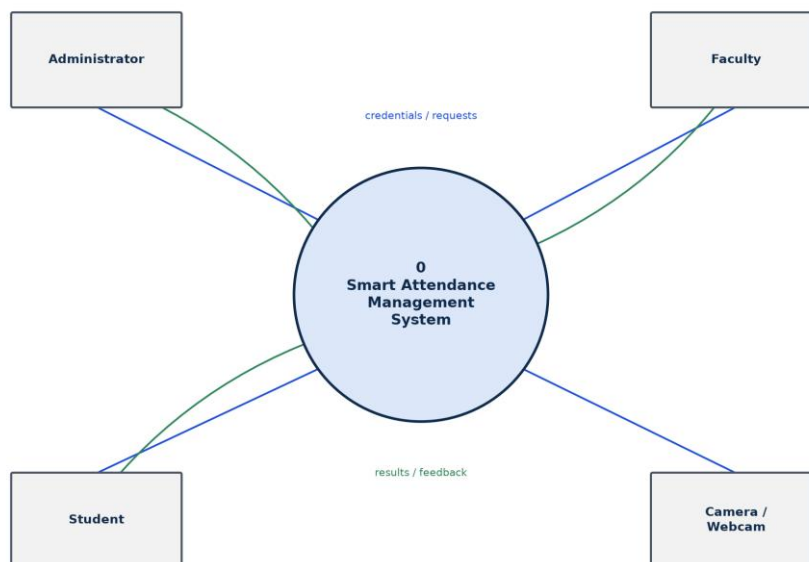


Fig. 4.2 Data Flow Diagram — Level 0 (Context)

Figure 4.3 decomposes the system into six first-level processes — authentication, academic-structure management, attendance marking (the union of the three routes described in Section 6.3), leave management, report/analytics generation, and audit/backup/settings — each reading from and writing to its own area of the database.

Fig. 4.3 Data Flow Diagram — Level 1

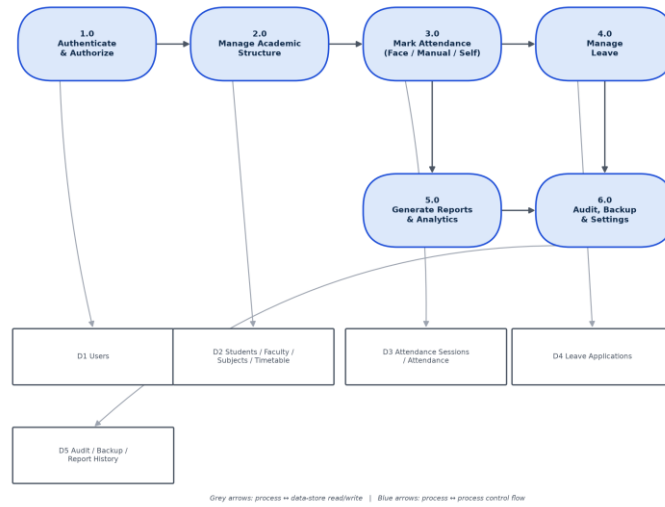


Fig. 4.3 Data Flow Diagram — Level 1

4.3 Use Case Diagram

Figure 4.4 summarises the sixteen use cases against the three actors. The Administrator's use-case set is a superset that additionally covers academic structure, timetable, leave override, audit trail, backup/restore and settings; Faculty is restricted to the classes and subjects assigned to them; a Student is restricted to their own attendance, leave and self-marking.

Fig. 4.5 Use Case Diagram

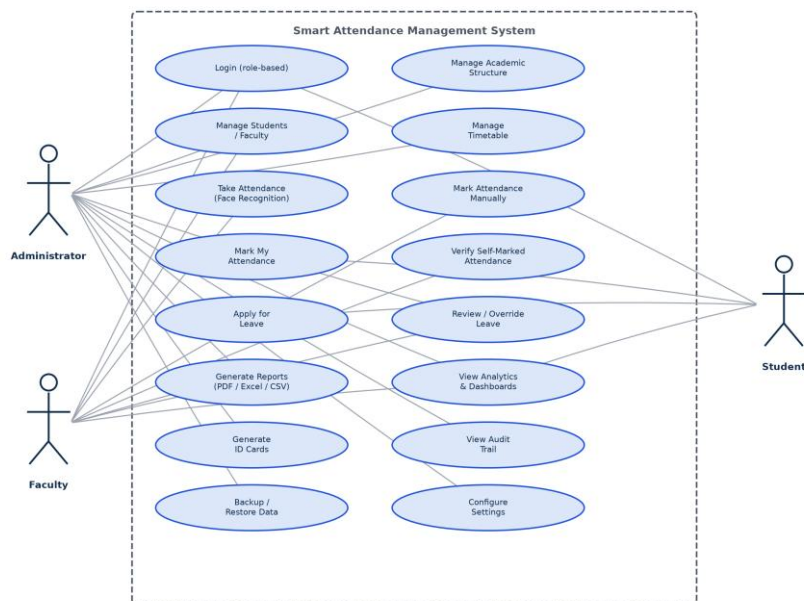


Fig. 4.4 Use Case Diagram

4.4 Database Design

The database is a single SQLite file with foreign keys enforced (PRAGMA foreign_keys = ON) and Write-Ahead Logging enabled, comprising 24 tables, 4 reporting views and 27 indexes. No image, document or backup file is stored as a database blob; the database stores only the file path, and the actual bytes live under storage/. Figure 4.5 shows the entity-relationship diagram for the core tables; the full table list, grouped by area, is given in Appendix A.

Fig. 4.4 Entity-Relationship Diagram (core entities)

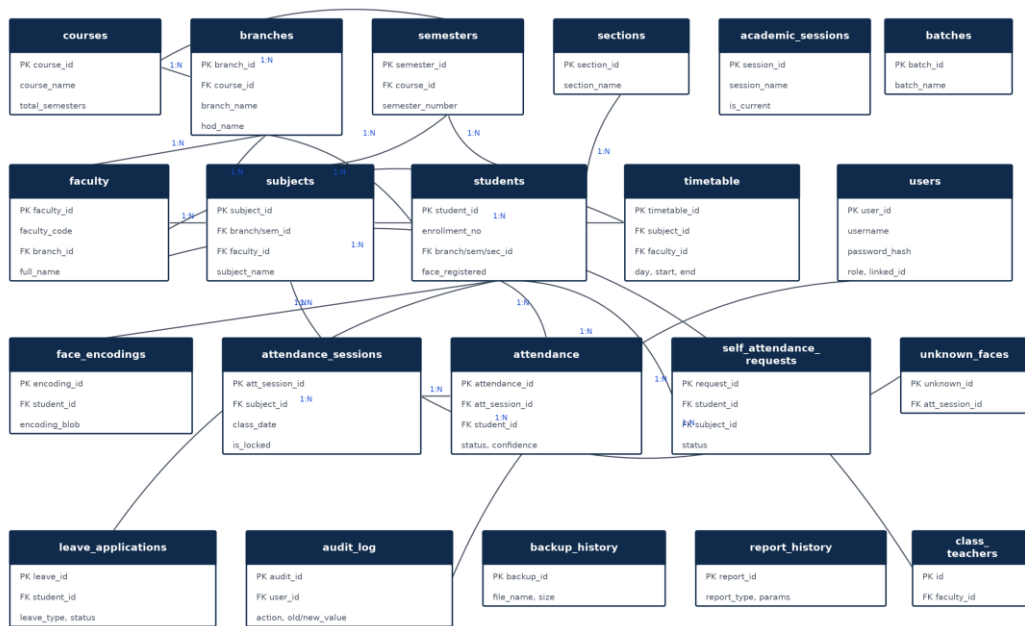


Fig. 4.5 Entity-Relationship Diagram (core entities)

Section	Tables
Authentication & people	users
Academic structure	courses, branches, semesters, sections, academic_sessions, batches
Faculty & subjects	faculty, subjects
Students	students, face_encodings, promotion_history
Timetable	timetable, class_teachers, holidays
Attendance	attendance_sessions, attendance, self_attendance_requests, unknown_faces
Leave	leave_applications

Section	Tables
Audit & system	audit_log, activity_log, backup_history, report_history
Reporting views	v_student_full, v_attendance_full, v_attendance_summary, v_timetable_full

Table 4.1: Database Table Groups

4.5 Key Architectural Decisions

A number of design choices were made deliberately, and are worth calling out because they are the reason several features behave the way they do:

- Attendance sessions are pre-seeded. Opening a face-recognition session immediately inserts an Absent row for every enrolled student in one transaction; recognising a face later UPDATES that row rather than inserting a new one. This single decision gives duplicate-mark prevention, “absentees for free”, and a clean, fully-auditable UPDATE trail, all from one modelling choice (Section 6.2).
- A student's own click is never attendance. Self-marking (`models/self_attendance.py`) only ever creates a Pending row; only a faculty approval mutates the attendance table, and the approving faculty's ownership of the subject is re-verified at approval time even though the UI queue is already filtered — the real security boundary is enforced in the model, not just the screen.
- Row-level scoping is centralised, not repeated. `core/scope.py` computes a SQL WHERE-clause fragment per role once per session and every model function that returns attendance, student or subject data splices it in, so a screen cannot accidentally leak another faculty member's class by forgetting a filter.
- The database runs in WAL mode. Because face-recognition attendance marking happens on a background thread while the UI keeps rendering on the main thread, Write-Ahead Logging lets a background write proceed without blocking a foreground read of the same database file.
- Two face-recognition backends share one interface. Whichever backend is active (Section 5.2), the rest of the application only ever sees a normalised 0–100% confidence score and a match/no-match decision, so nothing above the service layer changes if the active backend changes.

CHAPTER 5: TECHNOLOGY STACK

5.1 Programming Language and GUI Framework

The application is written entirely in Python 3, chosen for the maturity of its computer-vision (OpenCV), data (pandas, NumPy) and reporting (ReportLab, OpenPyXL) ecosystems, all of which the project draws on directly. The interface is built with CustomTkinter, a modern styling layer over the standard tkinter toolkit that ships with Python, supplemented by tkcalendar for date-picker widgets (used for date-of-birth, leave dates and report filters) and standard ttk widgets (Treeview) for data tables.

5.2 Face Recognition Backends

Two interchangeable recognition backends are supported, selected automatically at start-up depending on what is importable in the running environment:

Aspect	dlib / face_recognition	OpenCV LBPH (default fallback)
Representation	128-dimension face embedding per image	Local Binary Pattern histograms over a shared model
Installation	Needs a C++ toolchain; no prebuilt wheels for Python 3.13+	Ships in opencv-contrib-python; no compiler needed
Accuracy	Highest, especially across pose/lighting variation	Good, sufficient for classroom-distance recognition
Training unit	Per-student mean embedding	One shared model retrained across all students
Confidence scale	Normalised to 0–100%, higher is better	Normalised to 0–100%, higher is better (native LBPH distance is lower-is-better and is inverted internally)

Table 5.2: Face Recognition Backend Comparison

Because opencv-contrib-python version 5.x removed the bundled Haar cascade files that face detection needs and dropped the contrib LBPH recogniser entirely, the dependency is deliberately pinned to opencv-contrib-python $\geq$ 4.10, $<$ 5, and the application additionally

ships its own copy of the cascade file under assets/ as a final fallback if the OpenCV-bundled copy cannot be found.

5.3 Supporting Libraries

Library	Purpose in this project
customtkinter	Themed desktop GUI framework (all screens, dialogs, cards)
tkcalendar	Date-picker widgets for DOB, leave dates and report filters
opencv-contrib-python	Face detection (Haar cascade) and the LBPH recognizer
Pillow	Image handling — photos, ID cards, splash/logo rendering
numpy	Array math behind face encodings and distance calculations
pandas	Reading/validating imported Excel and CSV data
matplotlib	All analytics charts, embedded into CustomTkinter via TkAgg
openpyxl	Excel template generation, import and export
reportlab	PDF report generation with the college letterhead
bcrypt	Password hashing (cost factor 12)

Table 5.1: Core Technology Stack

5.4 Database

SQLite was chosen over a client-server database (PostgreSQL/MySQL) because the application is explicitly single-machine and offline; SQLite needs no separate server process, ships with Python, and its native `Connection.backup()` API allows a fully consistent snapshot backup to be taken even while the database is open in WAL mode — exactly the backup mechanism `core/backup.py` relies on (Section 6.5).

5.5 Development and Version Control

The project is version-controlled with Git, with a GitHub Actions workflow configured for continuous integration on pull requests. Source is organised as a conventional Python package with a clear one-module-per-responsibility layout, described fully in Chapter 6.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation Summary

The implementation spans 56 Python files and approximately 23,000 lines of code, organised strictly along the layering introduced in Section 4.1:

Layer	Location	Files	Approx. LOC	Responsibility
Core / infrastructure	core/, config/	10	2,650	Database, auth, audit, backup, validators, scope, seed, settings, theme
Business logic	models/	8	3,908	Academic, student, faculty, subject, timetable, attendance, self-attendance, leave
Services	services/	5	3,139	Face recognition, analytics, reports, ID cards, import/export
Presentation — shell	ui/	3	1,081	Splash, login, main app window / navigation
Presentation — screens	ui/views/	16	10,232	One module per screen (dashboard through settings)
Presentation — widgets	ui/widgets/	3	1,659	Reusable cards, dialogs, styled data table
Tests	tests/	3	96	Smoke tests: schema creation, module imports, validators

Table 6.1: Module-wise Implementation Summary

6.2 The Face Recognition Pipeline

Face recognition is implemented in `services/face_service.py` end to end: dataset capture, training and live recognition. Dataset capture runs on a worker thread so the interface never blocks; it targets 60 images per student by default, rejecting blurred frames using a Laplacian-variance sharpness gate (threshold 45.0) and spacing captures at least 0.12 seconds apart to get varied poses. Captured faces are normalised to 200×200 greyscale, histogram-equalised crops. Before a new dataset is saved, the system samples roughly eight of the captured images and

checks them against every other registered student's stored encoding, to catch accidental double-registration.

Training differs by backend: the dlib path computes the mean of each image's 128-dimension encoding (more robust to one bad frame than any single sample) and stores it against (student, backend); the LBPH path fits one shared model across every registered student and must be retrained whenever a new student is added, persisted to `trained_models/lbph_model.yml` with a pickled label map. During live recognition, the dlib path accepts a match only if the embedding distance is within a configurable tolerance (default 0.45) and the resulting confidence is at least the configured minimum (default 55%); the LBPH path inverts its native lower-is-better distance onto the same 0–100% scale before the same threshold is applied, so the rest of the application never needs to know which backend produced the score.

Fig. 6.1 Sequence Diagram — Face-Recognition Attendance

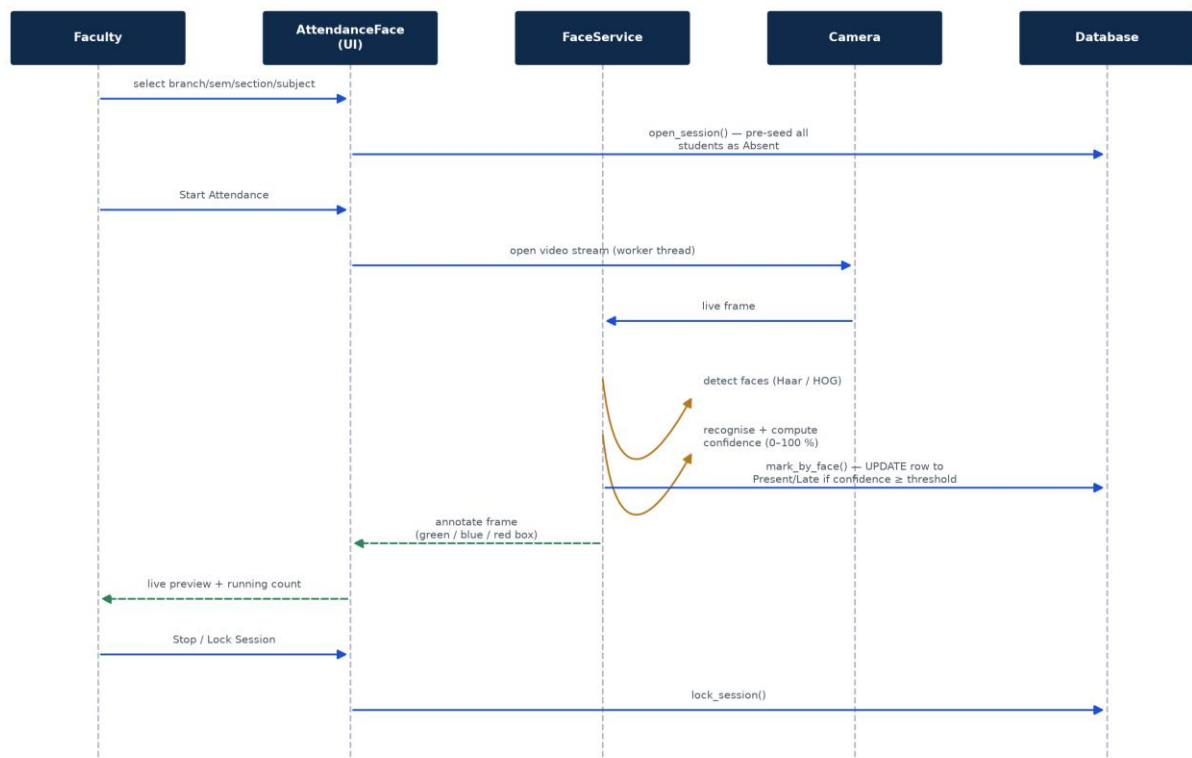
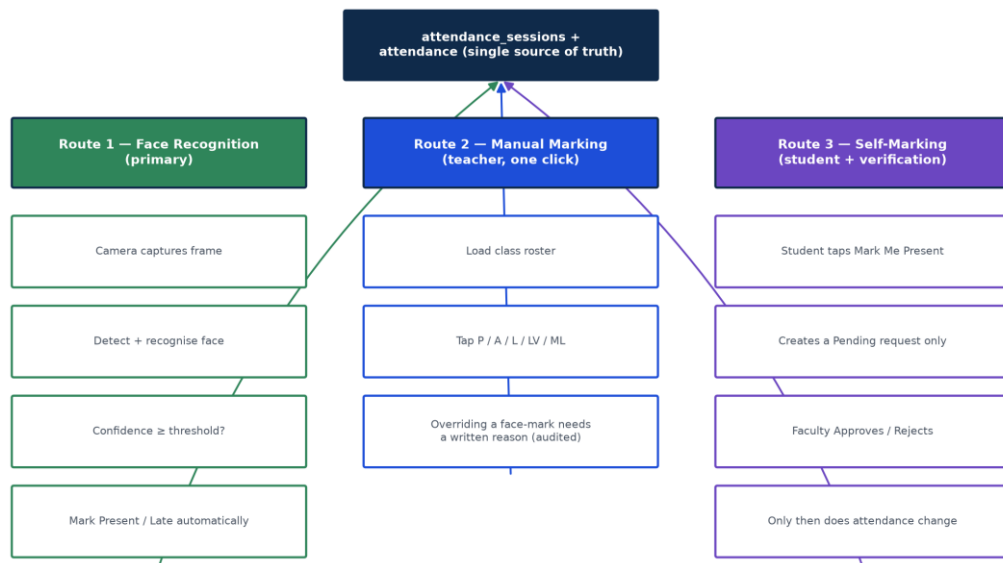


Fig. 6.1 Sequence Diagram — Face-Recognition Attendance

6.3 The Three Attendance Routes

A deliberate design decision of this project is that face recognition is the primary attendance route but never the only one. All three routes write to the same attendance table, and the (att_session_id, student_id) uniqueness constraint at the database level is what actually prevents a student from being marked twice, regardless of which route did it.

Fig. 6.3 Three Complementary Attendance Routes



All three routes converge on the same attendance record — one physical row per (session, student), enforced by a UNIQUE database constraint.

Fig. 6.3 Three Complementary Attendance Routes

6.3.1 Route 1 — Face Recognition (primary)

The teacher selects branch, semester, section and subject and starts the session; the camera opens on a worker thread, detects and recognises faces continuously, and marks each recognised student Present (or Late, if the arrival time is more than a configurable number of minutes — default 10 — after the scheduled start) the moment their confidence clears the threshold. The live preview draws a green box around a student just marked, blue around one already marked, and red around an unrecognised face.

6.3.2 Route 2 — Manual Marking

The teacher can load the class roster and mark each student Present, Absent, Late, Leave or Medical Leave with a single tap, plus an All-Present/All-Absent shortcut for the whole class.

Overriding a mark that face recognition already set requires a non-empty, audited reason — the system does not let a face-recognised Present silently become Absent without a trace of who changed it and why.

6.3.3 Route 3 — Self-Marking with Verification

A student can tap “Mark Me Present” for a class currently in session, but this only raises a Pending request — it is never, by itself, an attendance record. The request appears in the owning faculty member's Verify Attendance queue, where it can be approved (Present), approved as Late, or rejected; rejecting leaves the attendance record completely untouched. If the teacher never opened a session for that class, approving one automatically opens it, so a teacher is never blocked from approving just because they had not yet started the session. Guardrails prevent duplicate requests, reject requests on holidays or after a locked session, and close the request window a configurable number of minutes (default 240) after the class ends.

Fig. 6.2 Sequence Diagram — Student Self-Marking with Verification

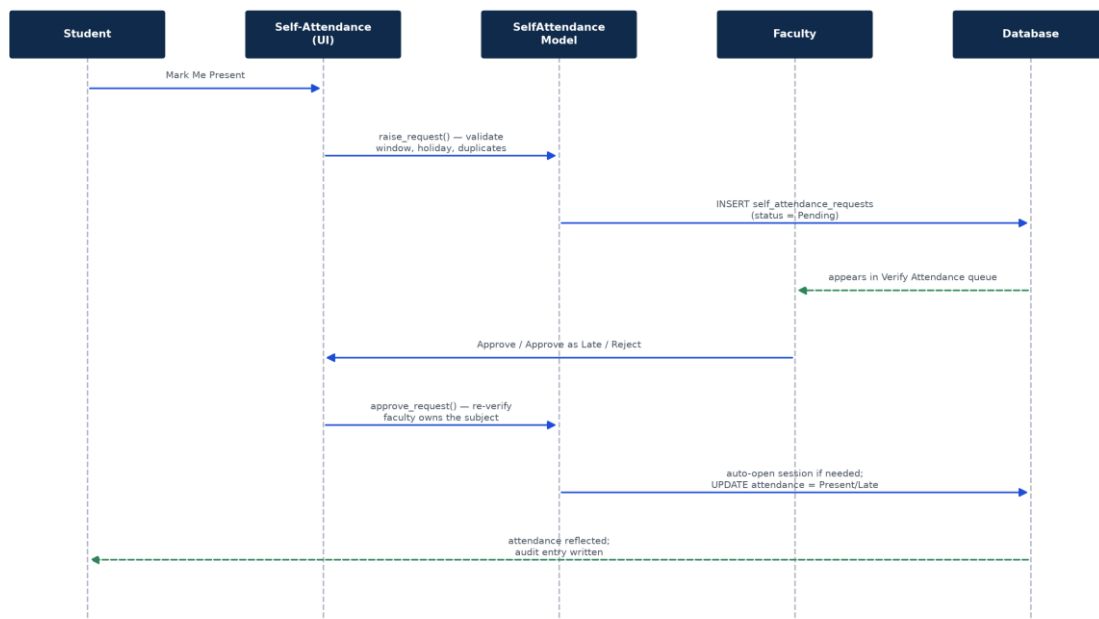


Fig. 6.2 Sequence Diagram — Student Self-Marking with Verification

6.4 Leave Management Workflow

A student applies for one of six leave types (Casual, Medical, Sports, Official, Academic, Other); medical leave beyond a configurable number of days (default 3) requires an uploaded medical certificate before it can be submitted. The application routes the request to that class's

class teacher specifically — not to any faculty member who happens to teach a subject there — and re-verifies that routing at approval time as the real security boundary, even though the review queue is already filtered by teacher. Approval updates the corresponding attendance rows to Leave or Medical Leave, but only for rows still marked Absent; a student who was physically recognised as Present on a date later covered by an approved leave keeps their Present mark. An Administrator can override any decision with a mandatory reason, and a student may withdraw a still-pending application.

6.5 Security Implementation

Passwords are hashed with bcrypt at cost factor 12. An account is locked for a configurable period after five consecutive failed attempts. Role-based access control is expressed as roughly two dozen named permission constants, each role mapped to the subset it holds — the Administrator role is granted every permission programmatically by introspecting the permission class, while Faculty and Student are each granted an explicit, deliberately restricted subset (for example, Faculty is not granted permission to unlock an attendance session or override a leave decision; only Administrator holds those). A representative slice of the permission matrix is shown below; the complete set spans academic, attendance, leave, reporting, audit and system-administration permissions.

Permission	Administrator	Faculty	Student
View own dashboard	Yes	Yes	Yes
Take attendance (face recognition)	Yes	Yes	No
Mark attendance manually	Yes	Yes	No
Mark own attendance (self-mark request)	No	No	Yes
Verify self-attendance requests	Yes	Yes	No
Lock an attendance session	Yes	Yes	No
Unlock an attendance session	Yes	No	No
Review leave applications	Yes	Yes	No
Override a leave decision	Yes	No	No
Manage academic structure / timetable	Yes	No	No
View audit trail	Yes	No	No

Permission	Administrator	Faculty	Student
Backup / restore database	Yes	No	No

Table 6.2: Role-wise Permission Matrix (representative)

Beyond navigation-level permissions, `core/scope.py` enforces row-level data scoping directly in the query layer: an Administrator's scope resolves to an unrestricted clause, a Faculty member's scope resolves to only the subjects and classes assigned to them, and a Student's scope resolves to only their own `student_id` — so the restriction holds even if a screen were to omit an explicit filter. Every sensitive action — attendance edits, leave overrides, lock/unlock, backups, restores, data import/export, password changes — is written to a dedicated `audit_log` table carrying the acting user, the action, the before/after values as JSON, and (for actions that require one) the reason given; a logging failure is itself logged but is never allowed to roll back the underlying business transaction.

6.6 Reports and Analytics

`services/report_service.py` generates twelve kinds of output — eleven analytical report types plus a printable, one-page attendance sheet for physical signature — in PDF (ReportLab), Excel (OpenPyXL) or CSV, every one carrying the same institutional header: college logo, name, address, the filters applied, the generation timestamp and the generating user's name. Attendance status cells are colour-coded consistently with the on-screen palette (Present/Absent/Late/Leave/Medical Leave), and every PDF closes with a three-column signature block for the class teacher, HOD and Principal.

Report / Output	Formats
Daily / Weekly / Monthly / Semester Attendance	PDF, Excel, CSV
Student / Subject / Branch / Faculty Attendance	PDF, Excel, CSV
Defaulter List (below attendance threshold)	PDF, Excel, CSV
Leave Report / Medical Leave Report	PDF, Excel, CSV
Printable Attendance Sheet (signature register)	PDF

Table 6.3: Report Catalogue

services/analytics_service.py renders the same data as matplotlib charts embedded directly into the CustomTkinter interface — daily/weekly/monthly trend lines, branch/subject/faculty/semester comparisons, a status-distribution donut, and an attendance-band histogram — every percentage chart drawing a threshold line at the configured eligibility percentage (default 75%) and colouring bars traffic-light style: green at or above threshold, amber within ten points below, red otherwise.

6.7 Representative User Interface

The screens below are live screen captures of the running application — taken across all three roles during actual use against the seeded Government Girls Polytechnic, Shamli data — rather than illustrative mockups, so every number, name and status shown is what the system itself produced.

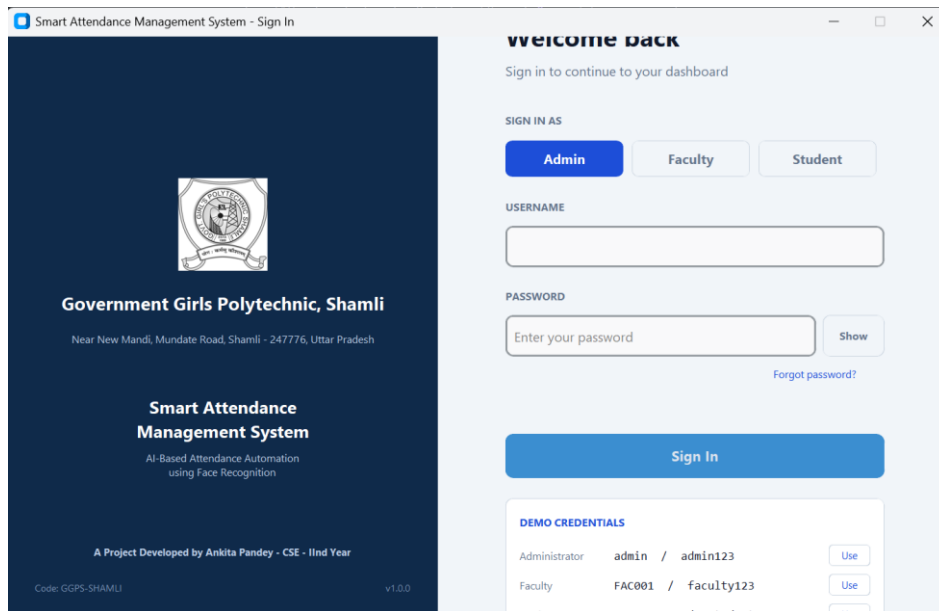


Fig. 6.4 Sign-In Screen

The sign-in screen requires an explicit role selection alongside the username and password; picking the wrong role for an otherwise-valid account is rejected, which is a deliberate access control rather than an oversight. A Demo Credentials panel queries real seeded accounts directly from the database — admin/admin123, FAC001/faculty123, DCSE25001/student123 — rather than hard-coded examples, with a one-click “Use” button per role.

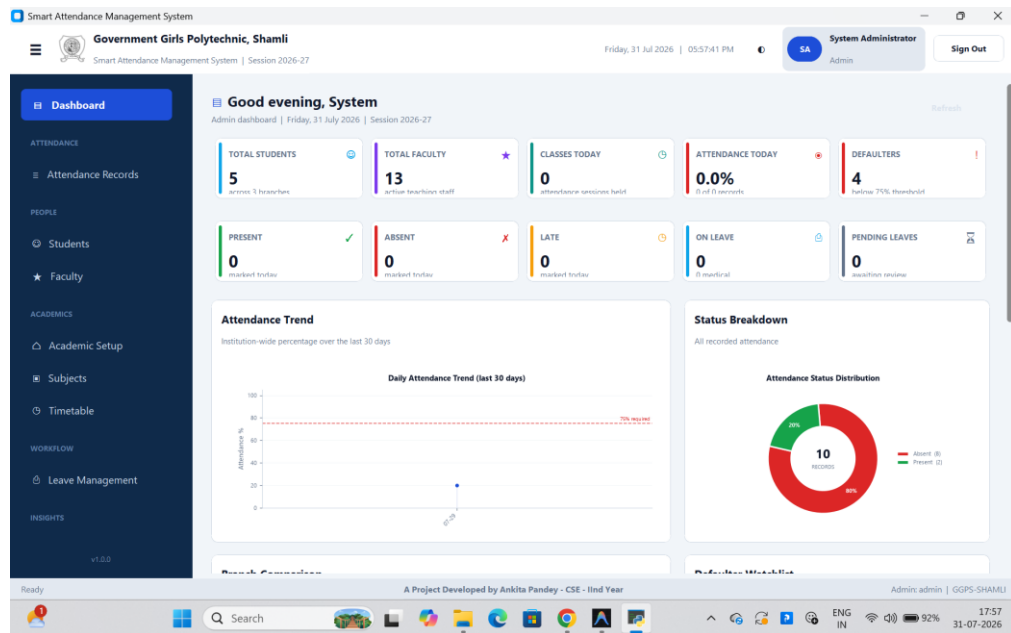


Fig. 6.5 Administrator Dashboard

The Administrator's dashboard shows institution-wide totals (students, faculty, classes held today), a 30-day attendance trend chart against the 75% threshold line, a status-distribution donut, and a defaulter watchlist below the fold — everything an office needs to see at a glance without opening a report.

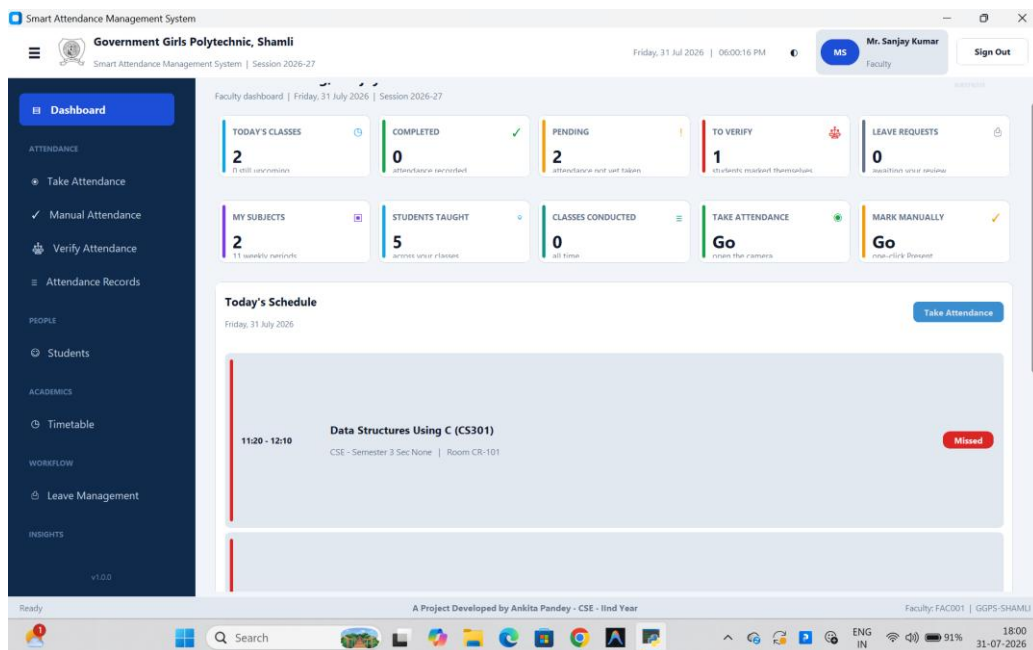


Fig. 6.6 Faculty Dashboard

A Faculty member instead sees exactly what their own day requires: today's classes, how many are completed versus still pending, how many self-attendance requests are awaiting their

verification, pending leave requests, and one-click shortcuts straight into Take Attendance or Mark Manually — visible here for Mr. Sanjay Kumar, HOD of Computer Science & Engineering.

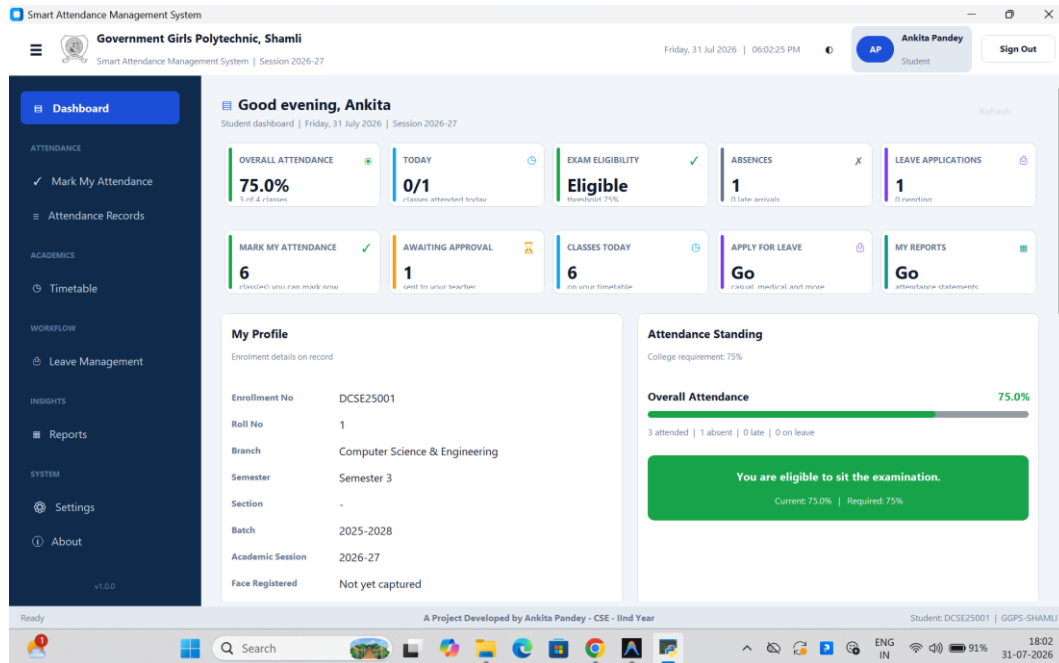


Fig. 6.7 Student Dashboard

A Student's dashboard (shown here for Ankita Pandey, DCSE25001) leads with the number that matters most to them — overall attendance percentage against the 75% requirement and a live exam-eligibility verdict — alongside today's classes, pending self-attendance requests, and shortcuts to mark attendance or apply for leave.

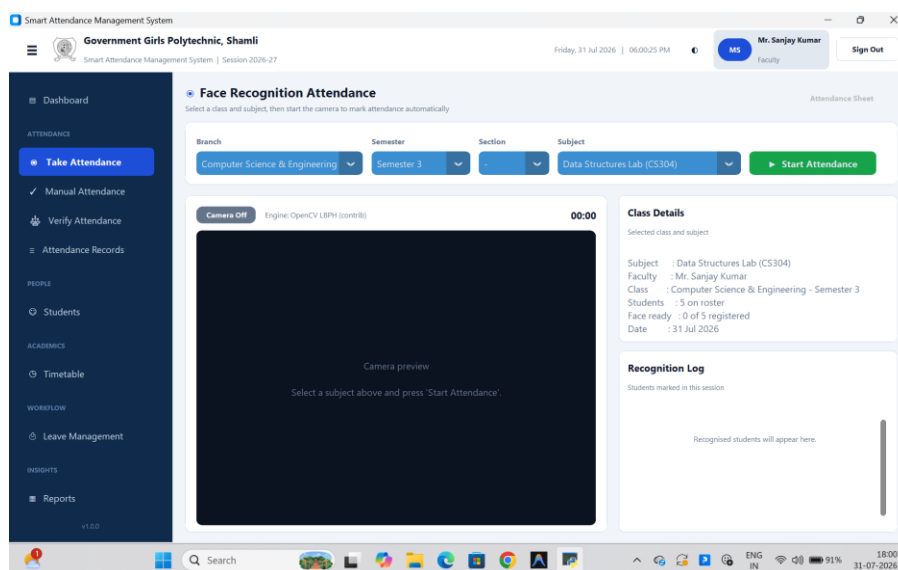


Fig. 6.8 Face-Recognition Attendance Screen

Take Attendance shows the class roster context (branch, semester, subject, how many of the 5 enrolled students already have a usable face dataset), the live camera preview, and a running recognition log — here shown before the camera is started, reporting the active backend as “OpenCV LBPH (contrib)”, the fallback engine this installation is actually running.

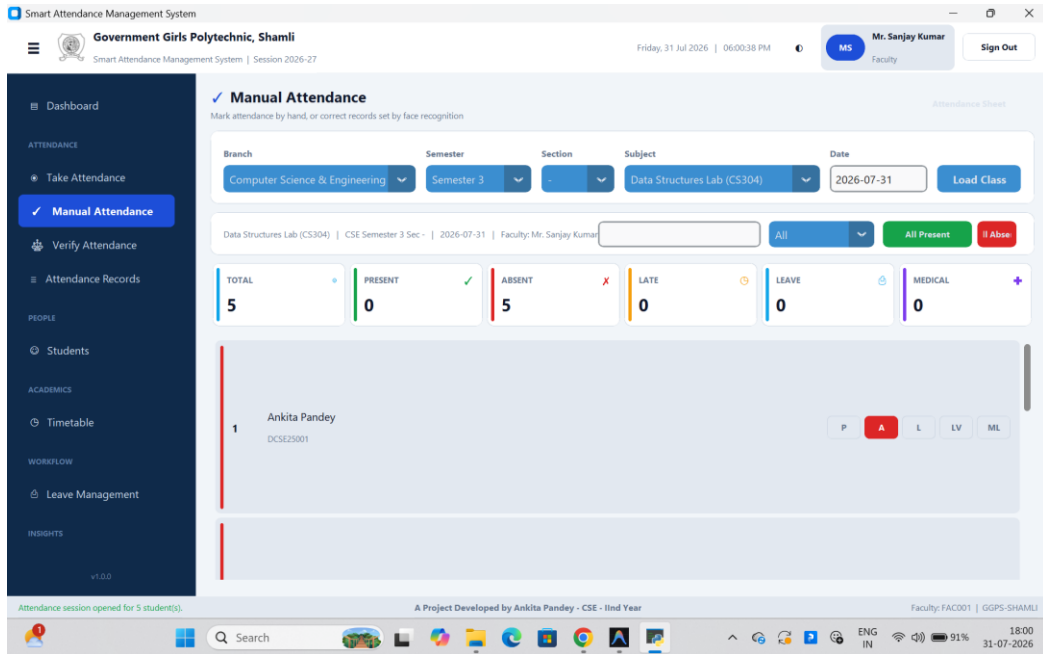


Fig. 6.9 Manual Attendance and Correction

Manual Attendance lists every enrolled student with one-click Present, Absent, Late, Leave, Medical-Leave buttons, plus All-Present, All-Absent shortcuts and live Total, Present, Absent, Late, Leave, Medical counters, shown here for Mr. Sanjay Kumar's Data Structures Lab session on 31 July 2026.

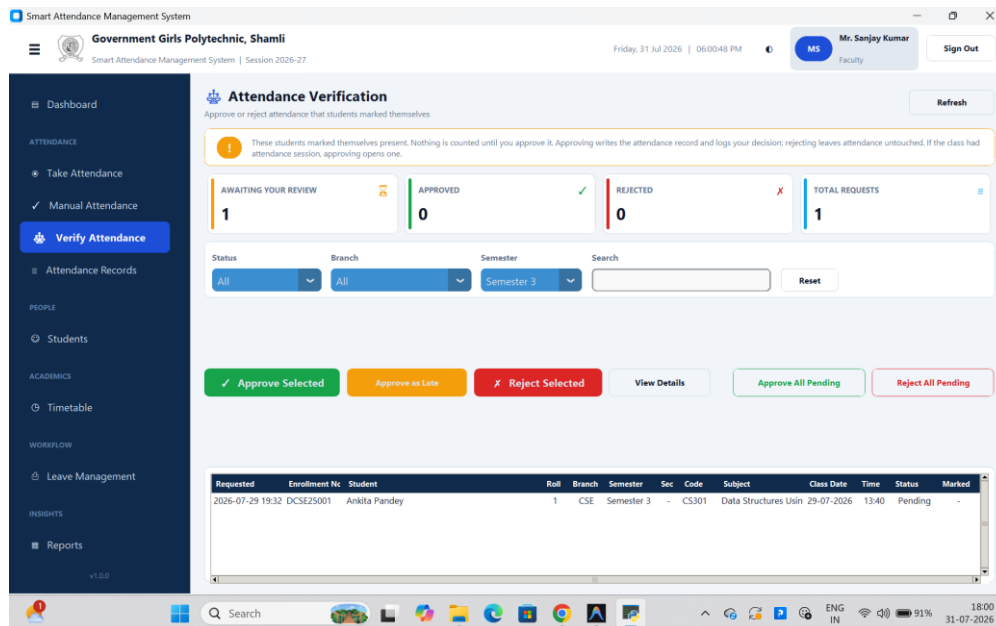


Fig. 6.10 Attendance Verification Queue

Verify Attendance is where a student's self-marked request actually becomes attendance: the banner states plainly that nothing is counted until the teacher acts, and Ankita Pandey's pending request for Data Structures Using C is queued here awaiting Mr. Sanjay Kumar's approval, reject, or approve-as-late decision.

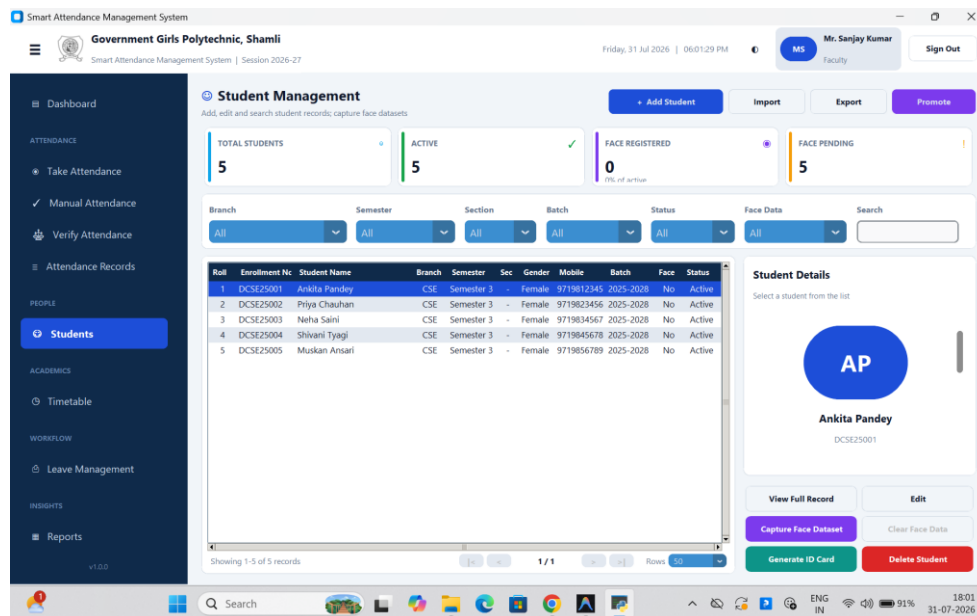


Fig. 6.11 Student Management Screen

The Students screen combines an add/edit form with a searchable, filterable data grid and exposes face-dataset capture and ID-card generation from the same toolbar — the seeded roster of five CSE Semester 3 students is shown here, all still pending face registration.

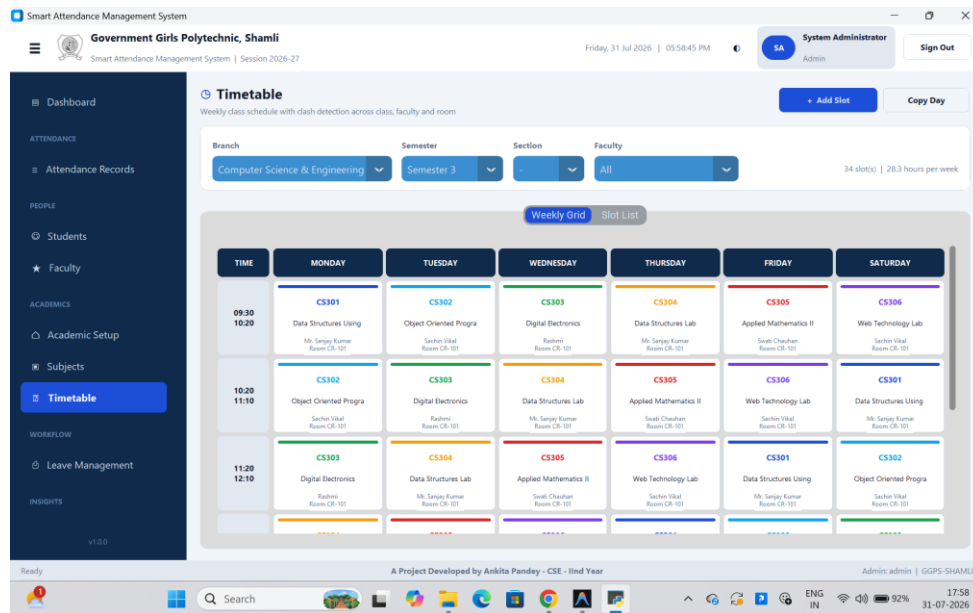


Fig. 6.12 Weekly Timetable

The Timetable screen renders the real weekly schedule generated for CSE Semester 3 — 34 slots totalling 28.3 teaching hours per week — colour-coding each subject consistently across the grid and reporting clash-free placement across Monday to Saturday.

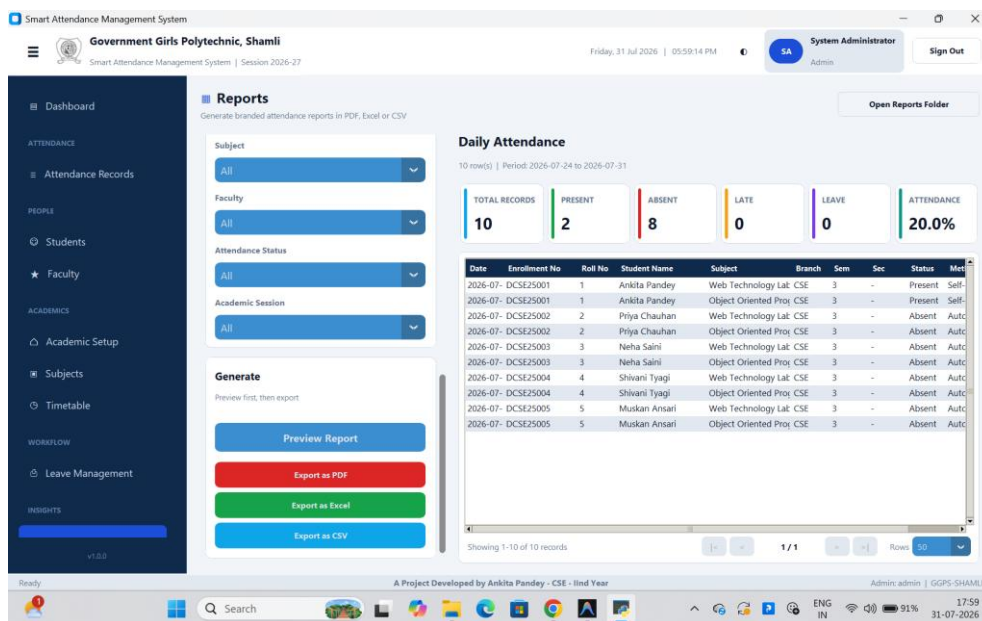


Fig. 6.13 Reports Screen

Reports lists all eleven report types on the left; selecting one loads a live preview before export. The Daily Attendance report shown here — generated for the period 24–31 July 2026 — reports 10 total records, 2 present and 8 absent, a 20.0% attendance rate for that window, ready to export as PDF, Excel or CSV.

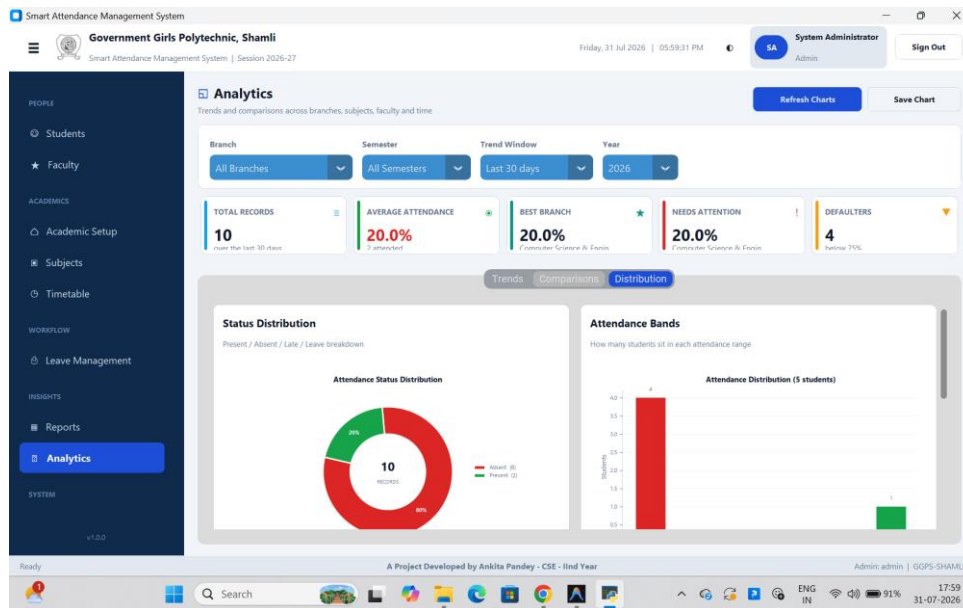


Fig. 6.14 Analytics Screen

Analytics tabs between trend, comparison and distribution views; the Distribution tab shown here renders the real status split (2 present, 8 absent — the same underlying 10 records as Figure 6.13) as a donut chart alongside a histogram of how many students fall into each attendance band.

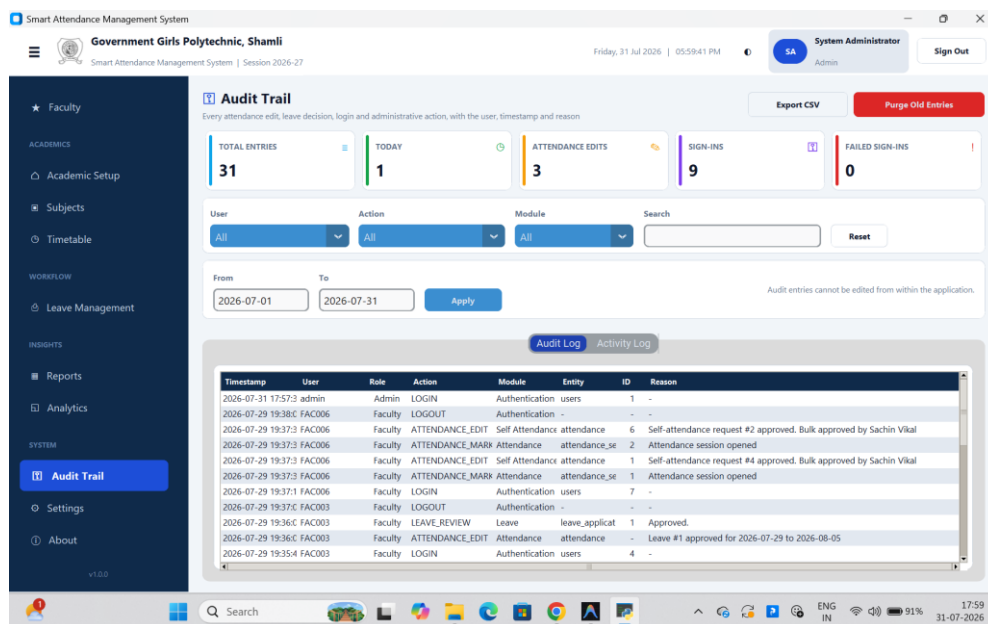


Fig. 6.15 Audit Trail

The Audit Trail is the tamper-evident log described in Section 6.5, made concrete: every login, logout, attendance edit, leave review and session-open action is listed with its exact timestamp, acting user, role, module, entity and — where one was required — the reason given, such as “Self-attendance request #2 approved. Bulk approved by Sachin Vikal”.

CHAPTER 7: TESTING

7.1 Testing Approach

Testing combined a small automated smoke-test suite with structured manual functional testing of every screen and workflow. The automated suite (tests/, 96 lines across three files) is intentionally a regression guard rather than full coverage: `test_database.py` verifies that a fresh schema initialises correctly and reports the expected tables; `test_imports.py` imports all seventeen core, config, model and service modules and asserts each one loads without error, catching syntax errors, circular imports and missing dependencies early; `test_validators.py` unit-tests the input validators (email, mobile, username, password strength) against valid and deliberately invalid input. Everything above the validators layer — the model business rules, the face-recognition pipeline, and every screen — was verified manually against the representative test cases below, since GUI and camera-dependent behaviour is difficult to assert automatically without a dedicated UI-testing harness, which is noted as a limitation in Section 8.3.

7.2 Representative Test Cases

ID	Test Case	Expected Result	Outcome
TC-01	Log in with a valid username/password and the correct role	Login succeeds; role-appropriate dashboard loads	Pass
TC-02	Log in with a valid password but the wrong role selected	Login rejected with a generic invalid-credentials message	Pass
TC-03	Fail login five times in a row for one account	Account is locked for the configured lockout period	Pass
TC-04	Recognise a face with confidence below the configured minimum	Student is not marked; face is logged to <code>unknown_faces</code>	Pass
TC-05	Recognise the same enrolled student twice in one session	Second recognition is a no-op; row stays marked once	Pass
TC-06	Manually change a face-marked Present to Absent without a reason	Save is blocked until a non-empty reason is provided	Pass
TC-07	Two timetable slots for the same faculty member overlap in time	Clash detected and slot save is rejected before it commits	Pass

ID	Test Case	Expected Result	Outcome
TC-08	Student raises a self-attendance request, then a second one for the same class/day	Second request is rejected as a duplicate	Pass
TC-09	Faculty approves a self-attendance request as Late	Attendance row updates to Late; action is audited	Pass
TC-10	Student attempts self-marking 5 hours after the class ends (grace window is 240 minutes)	Request is blocked as outside the marking window	Pass
TC-11	Apply for Medical Leave of 5 days without a certificate	Submission is blocked; certificate required beyond 3 days	Pass
TC-12	Administrator overrides a rejected leave to Approved	Leave and linked attendance rows update; reason is mandatory	Pass
TC-13	Generate a Defaulter report for a branch/semester	PDF lists only students below the eligibility threshold, with letterhead	Pass
TC-14	Restore a database backup file that is not a valid attendance database	Restore is rejected before any data is overwritten	Pass
TC-15	Import a student Excel sheet with one row missing Roll No	Only the invalid row is rejected, with its row number and reason; valid rows import	Pass
TC-16	Faculty attempts to view another faculty member's class list via direct data access	Row-level scope restricts the result to their own classes	Pass

Table 7.1: Representative Test Cases

7.3 Automated Smoke-Test Suite

The three automated test files were run with pytest against a temporary, disposable database file so that no test run could ever touch the live seeded database. All three passed on the final build used for this report, confirming the schema initialises cleanly, every core/config/model/service module imports without side effects, and the validator functions behave correctly at their documented boundaries (e.g., a mobile number is only accepted if it is exactly ten digits and begins with 6–9, matching the Indian numbering plan).

7.4 Results Summary

All sixteen representative test cases in Table 7.1 passed on the build used for this report. Combined with the automated smoke-test suite, this gives confidence that the security boundaries (lockout, role/route mismatch, mandatory reasons, row-level scope), the data-integrity boundaries (duplicate-mark prevention, clash detection, certificate enforcement) and the day-to-day reporting/import paths all behave as specified. Section 8.3 discusses the honest limits of this testing effort — principally, that it does not include an automated UI-level test harness.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Screens Delivered

Sixteen distinct screens were implemented and are reachable from the main navigation, filtered by the signed-in user's role: Dashboard, Take Attendance (face recognition), Manual Attendance, Mark/Verify Attendance (self-marking), Attendance Records, Students, Faculty, Subjects, Timetable, Leave Management, Reports, Analytics, Audit Trail, Settings and About. Section 6.7 shows live screen captures of twelve of these screens across all three roles; the remaining four follow the same card, table and dialog conventions shown throughout that section.

8.2 Achievements Against Objectives

Objective (Section 1.3)	Achieved As
Face-recognition pipeline on ordinary hardware	Dual-backend recognizer (dlib / OpenCV LBPH) auto-selected at start-up; runs on a standard webcam
Proxy attendance made structurally difficult	Live-camera recognition plus a database uniqueness constraint on (session, student)
Manual marking/correction that never fights automation	One-click P/A/L/LV/ML marking; overriding a face-mark requires a written, audited reason
Student self-marking never counts on its own	Self-attendance requests are Pending until a faculty approval mutates the record
Academic structure modelled as data	Courses/branches/semesters/sections/subjects/timetable fully editable, no hard-coded constants
Role-based access with row-level scoping	~24 named permissions per role plus core.scope query-level restriction
Institution-grade reporting	11 report types + printable attendance sheet, PDF/Excel/CSV, with letterhead and signatures
Tamper-evident audit trail	audit_log with before/after values and mandatory reasons on sensitive actions

Table 9.1: Objectives vs. Achievements

8.3 Limitations

- Automated test coverage is a smoke-test layer (schema, imports, validators) rather than full unit/integration coverage of the model and service layers; correctness above the validators was established through manual functional testing (Chapter 7), not an automated UI harness.
- Face recognition accuracy depends on classroom lighting and camera angle; the system does not currently perform liveness detection, so a sufficiently good printed photograph could in principle be presented to the camera — addressed as future work in Section 9.2.
- The system is single-institution and offline by design; it does not currently synchronise attendance data across multiple installations or campuses.
- The higher-accuracy dlib/face_recognition backend cannot be installed on Python 3.13+ without a working C++ toolchain, so most classroom PCs will run on the OpenCV LBPH fallback, which is slightly less accurate across extreme pose variation.
- There is no mobile companion application; all interaction happens on the desktop application installed on a college PC.

8.4 Discussion

Measured against the comparison in Table 2.1, the system meets its central design goal: attendance is proxy-resistant, the extra hardware requirement is a webcam most machines already have, and it works entirely offline. The three-route design (Section 6.3) proved to be the right shape for a real institution rather than a purely automated one — it gives the system a graceful way to handle the cases (poor lighting, a student not yet in the face dataset, a forgotten camera step) that any single-route attendance system would otherwise simply fail on. Centralising row-level security in `core/scope.py` rather than in each screen, and centralising the audit log's mandatory-reason enforcement in the model layer rather than the UI, means these guarantees hold even as new screens are added — a property that would be much harder to maintain if each screen were responsible for enforcing them independently.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project set out to replace paper-based attendance at Government Girls Polytechnic, Shamli with a system that is faster and harder to defeat than a register, while remaining faithful to how a real college actually operates — manual correction, student self-reporting under supervision, leave, promotion, timetable clashes and audit requirements all included. The result is a working, 23,000-line desktop application with a 24-table relational schema, dual-backend face recognition that degrades gracefully to hardware every college already owns, and a security model — bcrypt hashing, role-based permissions, row-level scoping and a tamper-evident audit trail — built to the standard a real institution, not just a classroom assignment, would require. Every objective set out in Section 1.3 was met, as summarised in Table 9.1, and the sixteen representative test cases in Chapter 7 passed against the final build.

Beyond the specific goal of attendance automation, the project demonstrates a broader point: an automation system for a real institution succeeds not by removing every manual process, but by giving each manual process a narrower, auditable role once the automatic path is doing most of the work.

9.2 Future Scope

- A companion mobile application for students to view attendance and apply for leave, communicating with the desktop installation over the college's local network rather than the public internet.
- Optional cloud synchronisation between branch campuses for institutions larger than a single-site polytechnic, without removing the fully-offline single-site mode this project targets.
- Liveness detection / anti-spoofing (blink detection or a challenge-response prompt) to close the printed-photograph gap noted in Section 8.3.
- SMS or email notifications to a parent or guardian when a student's attendance crosses below the eligibility threshold or a leave application is decided.
- Automatic timetable-driven session opening, so a scheduled class's attendance session opens itself at the scheduled time rather than requiring the teacher to start it manually.

- Adopting the dlib/face_recognition backend as the default once prebuilt wheels are available for the Python version in general campus use, without any change required above the service layer given the existing dual-backend abstraction.
- An automated UI-level test harness to close the coverage gap identified in Section 8.3.

REFERENCES

- [1] Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face description with local binary patterns: Application to face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12), 2037–2041.
- [2] Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [3] Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*.
- [4] OpenCV documentation — Face Recognition and Cascade Classification modules. <https://docs.opencv.org/>
- [5] Python Software Foundation. Python 3 documentation, including the tkinter and sqlite3 standard-library modules. <https://docs.python.org/3/>
- [6] SQLite documentation — Write-Ahead Logging and the Online Backup API. <https://www.sqlite.org/>
- [7] ReportLab documentation — PDF generation toolkit for Python. <https://www.reportlab.com/>
- [8] The pandas development team. pandas documentation. <https://pandas.pydata.org/>
- [9] TomSchimansky. CustomTkinter documentation. <https://github.com/TomSchimansky/CustomTkinter>
- [10] Geitgey, A. face_recognition — a simple Python face recognition library built on dlib. https://github.com/ageitgey/face_recognition
- [11] King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10, 1755–1758.
- [12] Government Girls Polytechnic, Shamli — official institutional website (course, branch and faculty information used for first-run seed data).

APPENDIX A: COMPLETE DATABASE SCHEMA

The complete schema (core/schema.sql) defines 24 tables, 4 reporting views and 27 indexes under PRAGMA foreign_keys = ON. Tables are listed below grouped exactly as they appear in the schema file, with their purpose.

Table	Purpose
users	Login accounts: username, bcrypt password hash, role, link to the faculty/student record, logout state
courses	Diploma programmes offered (e.g. the 3-year, 6-semester Diploma)
branches	Engineering branches under a course (CSE, Electronics, Fashion Designing)
semesters	Semesters generated automatically for each course
sections	Optional class subdivisions within a branch/semester
academic_sessions	Academic years (e.g. 2026-27), exactly one marked current
batches	Admission-year batches (e.g. 2026-2029)
faculty	Teaching and non-teaching staff records
subjects	Subjects, tied to course → branch → semester → assigned faculty
students	Student records, tied to branch/semester/section/batch/session
face_encodings	One stored face encoding per (student, recognition backend)
promotion_history	Record of each bulk semester promotion
timetable	Weekly class slots: day, time, subject, faculty, room
class_teachers	The one faculty member responsible for a class (routes leave approvals)
holidays	Institutional holiday calendar
attendance_sessions	One row per class actually conducted (subject/section/date/time), lockable
attendance	One row per (session, student); the attendance record itself
self_attendance_requests	Pending/approved/rejected student self-marking requests
unknown_faces	Faces detected during a session but not matched to any student
leave_applications	Student leave applications and their review status

Table	Purpose
audit_log	Tamper-evident log of sensitive actions with before/after values
activity_log	Low-severity activity log (logins, navigation) for support use
backup_history	Record of every database backup taken
report_history	Record of every report generated, with its filter parameters

Four reporting views resolve foreign keys to human-readable names for use throughout the reporting and analytics layers: v_student_full, v_attendance_full (the workhorse behind almost every report), v_attendance_summary (per-student-per-subject aggregated totals) and v_timetable_full.

APPENDIX B: REPRESENTATIVE CODE EXCERPTS

The excerpts below are simplified from the project's own source to illustrate the logic described in Chapter 6; they are not verbatim reproductions of the full implementation.

B.1 Normalised Confidence Scoring (services/face_service.py)

```
def confidence_from_distance(distance, backend, tolerance=0.45):
    if backend == "dlib":
        # Smaller distance = better match; normalise to 0-100, higher = better.
        conf = max(0, min(100, (1 - distance / (tolerance * 2)) * 100))
    else: # OpenCV LBPH: native distance is lower-is-better, invert it.
        conf = max(0, min(100, 100 - distance * 0.65))
    return conf
```

B.2 Pre-Seeding an Attendance Session (models/attendance.py)

```
def open_session(subject_id, section_id, class_date, roster, ...):
    session_id = db.insert("attendance_sessions", {...})
    # Every enrolled student starts Absent; recognition/marking
    # later UPDATES this row rather than inserting a new one.
    for student_id in roster:
        db.insert("attendance", {
            "att_session_id": session_id,
            "student_id": student_id,
            "status": "Absent",
        })
    return session_id
```

B.3 Role-Based Permission Check (core/auth.py)

```
class Permission:
    TAKE_ATTENDANCE = "take_attendance"
    UNLOCK_ATTENDANCE = "unlock_attendance"
    OVERRIDE_LEAVE = "override_leave"
    # ... ~24 named permissions in total

ROLE_PERMISSIONS = {
    "Admin": _all_permissions(),
    "Faculty": {Permission.TAKE_ATTENDANCE, Permission.VERIFY_SELF_ATTENDANCE, ...},
    "Student": {Permission.VIEW_DASHBOARD, Permission.MARK_SELF_ATTENDANCE, ...},
}

def require(permission):
    if not session.can(permission):
        raise PermissionError(f"Missing permission: {permission}")
```

B.4 Self-Marking Never Mutates Attendance Directly (models/self_attendance.py)

```
def raise_request(student_id, subject_id, ...):
    # A student's own click is never attendance -- it only
```

```

# ever creates a Pending row for a faculty member to act on.
return db.insert("self_attendance_requests", {
    "student_id": student_id,
    "subject_id": subject_id,
    "status": "Pending",
    ...
})

def approve_request(request_id, reviewer_id, mark_late=False):
    request = get_request(request_id)
    # Re-verify the reviewer actually owns this subject/class --
    # the real security boundary, not just the UI's filtered queue.
    require_owns_subject(reviewer_id, request.subject_id)
    mark_present_or_late(request, mark_late)

```

APPENDIX C: GLOSSARY OF TERMS

Term	Meaning in this project
Attendance session	One conducted class (a specific subject, section and date/time) opened for marking; the parent row of every individual attendance mark taken during it
LBPH	Local Binary Pattern Histogram — the OpenCV face-recognition algorithm used as the no-compiler fallback backend
Confidence score	A 0–100% score (higher is better on both recognition backends) indicating how closely a detected face matches a stored student encoding
Row-level scope	A restriction applied inside a database query — not just hidden in the interface — that limits which rows a given user's role is allowed to see or change
RBAC	Role-Based Access Control — granting or denying an action based on the signed-in user's role (Administrator / Faculty / Student)
Audit trail	The permanent, append-only record of every sensitive action, including who performed it, when, the before/after values, and the reason given
Class teacher	The one faculty member formally responsible for a specific branch/semester/section, who routes and approves that class's leave applications
Self-attendance request	A student-raised request to be marked present for a class, which has no effect on the attendance record until a faculty member approves it
Defaulter	A student whose attendance percentage has fallen below the configured eligibility threshold (default 75%)
WAL mode	SQLite's Write-Ahead Logging journal mode, which allows a background write and a foreground read to proceed without blocking each other
Face dataset	The set of captured face images (and the resulting trained encoding) stored for one student, used to recognise them during attendance
Seed data	The real institutional data (courses, branches, faculty, holidays) loaded automatically the first time the application runs against an empty database